

GPUMODE Lecture Study Notes: Lecture 1 How to Profile CUDA Kernels in PyTorch

Contents

1 Core Problem the Lecture Addresses

The lecture addresses a practical systems problem:

How can a PyTorch programmer move from ordinary tensor code to custom CUDA or Triton kernels, profile those kernels correctly, and decide when low-level GPU programming is worth the complexity?

The speaker’s main concern is not to teach CUDA from first principles in the traditional text-book order. Instead, the lecture is aimed at PyTorch programmers who are tired of what the speaker calls **CUDA tutorial hell**: repeatedly reading introductory CUDA tutorials without knowing how to use the material inside an actual PyTorch workflow.

The lecture focuses on the following workflow:

1. Start with a simple PyTorch operation such as elementwise squaring.
2. Profile the PyTorch operation to understand which CPU calls and CUDA kernels are actually executed.
3. Learn why naive Python timing is wrong for CUDA programs because CUDA execution is asynchronous.
4. Use PyTorch’s autograd profiler and visual profiler to inspect kernel launches and CPU-to-GPU data movement.
5. Integrate custom C++ and CUDA kernels into PyTorch using `torch.utils.cpp_extension.load_inline`.
6. Compare native PyTorch, custom CUDA, Triton, and `torch.compile`.
7. Use NVIDIA Nsight Compute, invoked through `ncu`, to understand occupancy, memory throughput, compute throughput, launch configuration, and actionable optimization hints.

The core technical trade-off is:

Developer productivity versus kernel-level performance control.

PyTorch gives productivity and composability. CUDA gives control over threads, memory, launch configuration, and low-level performance. Triton is positioned as a middle point: easier than CUDA, still much closer to the hardware than ordinary PyTorch.

2 Required Background

2.1 PyTorch Execution Model

A PyTorch tensor operation such as

```
b = torch.square(a)
```

looks like a single Python function call, but internally it may involve several layers:

1. Python-level function dispatch.
2. PyTorch C++ dispatcher.

3. ATen operator invocation.
4. CUDA runtime API calls.
5. CUDA kernel launch.
6. GPU execution on streaming multiprocessors.

The lecture emphasizes that profiling reveals these hidden layers. For example, `torch.square(a)` may internally map to an ATen power operation rather than to a distinct specialized square operation. Meanwhile, `a * a` may dispatch to a multiplication kernel and be slightly faster.

A useful mental model is:

PyTorch Python call $\rightarrow$ ATen C++ operator $\rightarrow$ CUDA runtime launch $\rightarrow$ GPU kernel.

2.2 CUDA Asynchrony

The most important CUDA profiling point in the lecture is that CUDA is asynchronous with respect to the CPU.

When Python launches a CUDA kernel, the CPU usually does not wait for the GPU to finish. Instead, it enqueues work onto a CUDA stream and continues executing.

Therefore, this timing code is misleading:

```
import time

start = time.time()
y = torch.square(x)
end = time.time()

print(end - start)
```

This often measures only the **kernel launch overhead**, not the actual GPU execution time. The correct timing model is:

$$T_{\text{measured incorrectly}} \approx T_{\text{CPU launch overhead}},$$

whereas the desired measurement is:

$$T_{\text{kernel}} = T_{\text{GPU finish}} - T_{\text{GPU start}}.$$

To measure GPU execution correctly, one must use CUDA events or synchronize the device before reading wall-clock time.

2.3 CUDA Events

CUDA events are timestamp-like objects recorded into a CUDA stream. They allow the programmer to measure elapsed GPU time between two points in the stream.

A simplified timing structure is:

```
start_event = torch.cuda.Event(enable_timing=True)
end_event = torch.cuda.Event(enable_timing=True)

start_event.record()
y = torch.square(x)
end_event.record()

torch.cuda.synchronize()
elapsed_ms = start_event.elapsed_time(end_event)
```

The synchronization is necessary because the CPU must wait until the GPU has completed the work before the elapsed time is meaningful.

2.4 Warmup

The first CUDA call in a process can include extra overhead:

1. CUDA context initialization.
2. Lazy library loading.
3. Memory allocator setup.
4. Kernel cache setup.
5. Potential JIT compilation in systems such as Triton or `torch.compile`.

Therefore, the lecture stresses the need for warmup iterations before measuring kernel timing. A common timing structure is:

$$T_{\text{reported}} = \frac{1}{N} \sum_{i=1}^N T_i,$$

where the measured iterations occur after warmup iterations have already run.

3 Main Concepts

3.1 The Toy Example: Elementwise Squaring

The central example is elementwise squaring of a tensor or matrix. Given an input tensor A , the output tensor B is

$$B_{i,j} = A_{i,j}^2.$$

Equivalently, if the tensor is flattened into a vector of length N ,

$$B_k = A_k \cdot A_k, \quad 0 \leq k < N.$$

This operation is simple, but it is ideal for profiling because it exposes important GPU performance concepts:

1. It is pointwise.

2. It has low arithmetic intensity.
3. It is usually memory-bandwidth sensitive.
4. It maps naturally to many independent GPU threads.
5. It is easy to compare across PyTorch, CUDA, Triton, and `torch.compile`.

For each element, the kernel performs one multiplication and moves data from and to memory. For FP32 data:

Bytes per element = 4 bytes read + 4 bytes written = 8 bytes.

The arithmetic intensity is approximately

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}} = \frac{1}{8} = 0.125 \text{ FLOPs/byte.}$$

This is very low. Therefore, the operation is unlikely to be compute-bound on modern GPUs. It is more likely to be memory-bound or launch-overhead-bound for small tensors.

3.2 Comparing `torch.square`, `a * a`, and `pow`

The lecture compares variants such as:

```
y1 = torch.square(a)
y2 = a * a
y3 = torch.pow(a, 2)
```

Profiling reveals that these may dispatch to different ATen operators:

- `torch.square(a)` may lower to an ATen power operation.
- `a * a` lowers to an elementwise multiplication operation.
- `torch.pow(a, 2)` directly invokes a power-style operation.

The key lesson is that two expressions that are mathematically equivalent may not generate the same low-level kernel.

Mathematically,

$$a^2 = a \cdot a,$$

but computationally,

$$\text{pow}(a, 2) \not\equiv \text{mul}(a, a)$$

in terms of dispatch, kernel implementation, and performance.

3.3 Pointwise Operations in Machine Learning

Elementwise square is used as a teaching example, but the same profiling ideas apply to many machine learning operations, including:

- activations,
- elementwise multiplication,
- elementwise addition,
- masking,
- normalization substeps,
- parts of softmax,
- trigonometric functions,
- scaling operations,
- bias addition,
- dropout masks.

Many deep learning models execute large numbers of small pointwise operations. If each operation launches a separate kernel, overhead and memory traffic can dominate.

For a chain of pointwise operations,

$$Y = f_3(f_2(f_1(X))),$$

eager PyTorch may launch one kernel per operation:

$$K_1, K_2, K_3.$$

A fused kernel can compute the full expression in one pass:

$$Y_i = f_3(f_2(f_1(X_i))).$$

The memory traffic difference is large. Without fusion, intermediate tensors are written to and read from global memory. With fusion, intermediate values can remain in registers.

3.4 Kernel Fusion

Kernel fusion is one of the most important optimization themes in the lecture.

Suppose we compute two squares:

```
a = torch.square(a)
a = torch.square(a)
```

Mathematically, this is:

$$a_{\text{out}} = (a_{\text{in}}^2)^2 = a_{\text{in}}^4.$$

In eager execution, PyTorch may launch two kernels:

$$K_1 : t_i = a_i^2, \quad K_2 : y_i = t_i^2.$$

This requires memory traffic for the intermediate tensor t :

$$\text{Traffic}_{\text{eager}} \approx \underbrace{4N}_{\text{read } a} + \underbrace{4N}_{\text{write } t} + \underbrace{4N}_{\text{read } t} + \underbrace{4N}_{\text{write } y} = 16N \text{ bytes.}$$

A fused kernel can compute:

$$y_i = (a_i \cdot a_i)(a_i \cdot a_i)$$

with traffic:

$$\text{Traffic}_{\text{fused}} \approx \underbrace{4N}_{\text{read } a} + \underbrace{4N}_{\text{write } y} = 8N \text{ bytes.}$$

Thus fusion can reduce memory traffic by approximately

$$\frac{16N}{8N} = 2.$$

For memory-bound pointwise operations, this can be a major speedup.

4 Profiling Tools Introduced in the Lecture

4.1 Tool 1: Correct Timing with CUDA Events

The first level of profiling is not a full profiler. It is simply correct timing.

```
import torch

def time_cuda_function(fn, *args, warmup=10, iters=100):
    for _ in range(warmup):
        fn(*args)

    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)

    start.record()
    for _ in range(iters):
        fn(*args)
    end.record()

    torch.cuda.synchronize()

    return start.elapsed_time(end) / iters
```

Line-by-line explanation:

- `warmup` runs the function several times before measurement to avoid first-use overhead.
- `torch.cuda.synchronize()` ensures all warmup work is complete.

- `torch.cuda.Event(enable_timing=True)` creates GPU-side timing events.
- `start.record()` records a start marker in the CUDA stream.
- The loop launches the operation repeatedly.
- `end.record()` records the end marker.
- The second synchronization waits for the GPU to finish.
- `elapsed_time` returns elapsed GPU time in milliseconds.

This avoids the classic mistake of measuring only CPU launch time.

4.2 Tool 2: PyTorch Autograd Profiler

The PyTorch autograd profiler gives a table of CPU and CUDA activity. It can show which PyTorch and ATen operations are executed and how much time they consume.

A representative usage pattern is:

```
import torch

x = torch.randn((1024, 1024), device="cuda")

with torch.autograd.profiler.profile(use_cuda=True) as prof:
    y = torch.square(x)

print(prof.key_averages().table(sort_by="cuda_time_total"))
```

The profiler can reveal that a high-level Python operation dispatches into lower-level ATen operators. For example, it can reveal whether a square operation uses a power kernel or a multiplication kernel.

This profiler is useful for answering:

1. Which PyTorch operators are actually being called?
2. How many times are they called?
3. How much CPU time is spent dispatching?
4. How much CUDA time is spent executing kernels?
5. Are there unexpected extra operations?

4.3 Tool 3: PyTorch Profiler and Chrome Trace

The PyTorch profiler can export a Chrome trace. This gives a visual timeline of CPU activity, CUDA runtime calls, memory copies, and GPU kernels.

Representative code:

```
import torch
from torch.profiler import profile, ProfilerActivity

x_cpu = torch.randn((1024, 1024))
```



```

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True
) as prof:
    x_gpu = x_cpu.to("cuda")
    y = torch.square(x_gpu)

prof.export_chrome_trace("trace.json")

```

The trace can be opened in Chrome tracing tools. The lecture uses this profiler to show operations such as:

- CPU-side PyTorch calls,
- `aten::copy_`,
- CUDA memory copy calls,
- `cudaMemcpyAsync`,
- host-to-device memory transfer,
- CUDA kernel launch events,
- GPU kernel execution.

4.4 Host-to-Device Copies

When a CPU tensor is moved to the GPU,

```
x_gpu = x_cpu.to("cuda")
```

the profiler may show a host-to-device memory copy. This corresponds to moving data from CPU RAM to GPU global memory.

The transfer can be modeled as:

$$T_{\text{copy}} \approx \frac{\text{Bytes transferred}}{\text{PCIe or NVLink bandwidth}} + T_{\text{copy overhead}}.$$

For a tensor with N FP32 elements,

$$\text{Bytes transferred} = 4N.$$

This is often much slower than moving data already resident on the GPU. Therefore, profiling must distinguish between:

time to transfer input to GPU

and

time to execute the GPU kernel.

4.5 Flow Events

The PyTorch visual profiler can show flow events linking CPU-side launch calls to GPU-side kernel execution.

The conceptual chain is:

`aten::square` $\rightarrow$ `aten::pow` $\rightarrow$ `cudaLaunchKernel` $\rightarrow$ GPU elementwise kernel.

This is important because it teaches the programmer to connect a high-level PyTorch expression to the actual kernel that ran on the GPU.

4.6 Tool 4: NVIDIA Nsight Compute Through `ncu`

The lecture presents NVIDIA Nsight Compute as the most useful CUDA profiler for low-level kernel optimization.

A basic command is:

```
ncu python train.py
```

A fuller report can be generated with:

```
ncu --set full -o report python train.py
```

Nsight Compute can report:

- L1 cache throughput,
- L2 cache throughput,
- DRAM throughput,
- compute throughput,
- memory bandwidth utilization,
- occupancy,
- launch statistics,
- warp stalls,
- tail effects,
- long scoreboard stalls,
- block size,
- grid size,
- source-level correlations.

The lecture emphasizes that `ncu` provides actionable optimization hints. For example, it may say that the kernel grid is too small to fill the GPU or that the achieved occupancy is low.

5 Hardware-Level Explanation

5.1 GPU Execution Hierarchy

A CUDA kernel is launched over a grid of thread blocks. Each block contains threads. Threads are executed on streaming multiprocessors, usually abbreviated as SMs.

The hierarchy is:

Grid $\rightarrow$ Thread blocks $\rightarrow$ Warps $\rightarrow$ Threads.

A warp is typically 32 threads:

warp size = 32.

The GPU schedules warps onto SMs. Each SM contains execution units, registers, shared memory, L1 cache, warp schedulers, and load/store units.

Visual Concept

Draw a grid split into many thread blocks. Then draw several SM boxes. Show thread blocks assigned to SMs, and inside each SM show warps, registers, shared memory, L1 cache, and CUDA cores.

5.2 Mapping Elementwise Square to CUDA Threads

For a flattened tensor of length N , a simple CUDA mapping is:

global thread id = blockIdx.x * blockDim.x + threadIdx.x.

Each thread handles one element:

$$\text{if } i < N, \quad B_i = A_i \cdot A_i.$$

A representative CUDA kernel is:

```
__global__ void square_kernel(const float* input, float* output, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        float x = input[idx];
        output[idx] = x * x;
    }
}
```

Explanation:

- `blockIdx.x` identifies the block within the grid.
- `blockDim.x` is the number of threads per block.
- `threadIdx.x` identifies the thread within the block.
- `idx` is the flattened element index.

- The boundary check prevents out-of-bounds memory access when N is not divisible by the block size.
- Each thread performs one global load, one multiply, and one global store.

5.3 Memory Coalescing

For good global memory performance, neighboring threads in a warp should access neighboring memory addresses.

In the simple square kernel, thread i accesses `input[i]` and `output[i]`. Therefore, the memory access pattern is contiguous:

$$\text{Thread } 0 \rightarrow A_0, \quad \text{Thread } 1 \rightarrow A_1, \quad \dots, \quad \text{Thread } 31 \rightarrow A_{31}.$$

This is a coalesced access pattern. The GPU can combine these loads and stores into efficient memory transactions.

5.4 Memory-Bound Behavior

For FP32 elementwise square, each element requires:

1 multiply

and approximately

8 bytes of global memory traffic.

The arithmetic intensity is:

$$I = \frac{1}{8} = 0.125 \text{ FLOPs/byte.}$$

Using a roofline-style model,

$$P \leq \min(P_{\text{peak compute}}, B_{\text{memory}} \cdot I),$$

where:

- P is achievable performance,
- $P_{\text{peak compute}}$ is peak GPU compute throughput,
- B_{memory} is memory bandwidth,
- I is arithmetic intensity.

Since I is small, the bound is likely:

$$P \approx B_{\text{memory}} \cdot I.$$

Thus, optimizing this operation is usually about memory traffic, launch overhead, fusion, and occupancy rather than raw floating-point throughput.

5.5 Occupancy

Occupancy describes how many warps are active on an SM relative to the maximum possible.

A simplified definition is:

$$\text{Occupancy} = \frac{\text{active warps per SM}}{\text{maximum warps per SM}}.$$

Low occupancy can occur when:

- the grid is too small,
- the block size is too small,
- each block uses too many registers,
- each block uses too much shared memory,
- the launch configuration does not expose enough parallelism.

The lecture highlights a case where Nsight Compute reports that the kernel grid is too small to fill the available GPU resources. This is a concrete example of insufficient parallel work.

5.6 Waves and Tail Effects

A wave is a batch of thread blocks that can occupy the GPU at once. If the number of blocks is not enough to fully populate all SMs, or if the final wave is only partially full, the GPU may be underutilized.

Let:

$$B_{\text{grid}} = \text{number of thread blocks in the grid},$$

$$S = \text{number of SMs},$$

and

$$B_{\text{per SM}} = \text{resident blocks per SM}.$$

The number of blocks that can run in one full wave is approximately:

$$B_{\text{wave}} = S \cdot B_{\text{per SM}}.$$

The number of waves is:

$$W = \left\lceil \frac{B_{\text{grid}}}{B_{\text{wave}}} \right\rceil.$$

If the last wave contains few blocks, the tail of execution underutilizes the GPU. Nsight Compute can report this as a tail effect.

6 Mathematical Reasoning

6.1 CUDA Indexing Math for a Flattened Tensor

For a tensor with shape $M \times N$, row-major linear indexing is:

$$k = i \cdot N + j,$$

where:

- i is the row index,
- j is the column index,
- N is the number of columns,
- k is the flattened index.

The inverse mapping is:

$$i = \left\lfloor \frac{k}{N} \right\rfloor, \quad j = k \bmod N.$$

For elementwise square:

$$B_{i,j} = A_{i,j}^2 = A_{i,j} \cdot A_{i,j}.$$

In flattened form:

$$B_k = A_k \cdot A_k.$$

6.2 Grid Size Calculation

If each CUDA thread processes one element, and each block has T threads, then the number of blocks required for N elements is:

$$B = \left\lceil \frac{N}{T} \right\rceil.$$

In integer C++ code, this is often written as:

$$B = \frac{N + T - 1}{T}.$$

Representative host-side launch code:

```
int threads = 256;
int blocks = (n + threads - 1) / threads;

square_kernel<<<blocks, threads>>>(input, output, n);
```

6.3 Launch Overhead Versus Kernel Work

For small tensors, the runtime can be dominated by launch overhead:

$$T_{\text{total}} = T_{\text{launch}} + T_{\text{kernel}}.$$

For elementwise square:

$$T_{\text{kernel}} \approx \frac{8N}{B_{\text{effective}}},$$

where $B_{\text{effective}}$ is effective memory bandwidth.

If N is small, then:

$$T_{\text{launch}} \gg T_{\text{kernel}}.$$

If N is large, then:

$$T_{\text{kernel}} \gg T_{\text{launch}}.$$

This explains why custom kernels and fusion matter especially when many small operations would otherwise each pay separate launch overhead.

6.4 Fusion Memory Traffic Model

For a sequence of m pointwise operations over N FP32 elements, if each operation writes an intermediate tensor and the next operation reads it, the memory traffic is roughly:

$$\text{Traffic}_{\text{unfused}} \approx 4N + 8N(m - 1) + 4N.$$

This includes:

- initial read,
- intermediate writes and reads,
- final write.

A fused implementation can often reduce this to:

$$\text{Traffic}_{\text{fused}} \approx 4N + 4N = 8N.$$

Thus, for large m , fusion can dramatically reduce memory traffic.

7 Code Walkthrough

7.1 Correct Benchmarking in PyTorch

A minimal benchmark for comparing square implementations is:

```

import torch

def square_builtin(x):
    return torch.square(x)

def square_mul(x):
    return x * x

def benchmark(fn, x, warmup=25, iters=100):
    for _ in range(warmup):
        fn(x)

    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)

    start.record()
    for _ in range(iters):
        fn(x)
    end.record()

    torch.cuda.synchronize()

    return start.elapsed_time(end) / iters

x = torch.randn((4096, 4096), device="cuda")

print("torch.square:", benchmark(square_builtin, x), "ms")
print("x * x:", benchmark(square_mul, x), "ms")

```

Important details:

- The tensor is allocated directly on the GPU, avoiding host-to-device copy during the timed region.
- Warmup removes one-time initialization effects.
- CUDA events measure GPU time.
- Synchronization ensures that timing is complete before reading the result.

7.2 Autograd Profiler Example

```

import torch

x = torch.randn((2048, 2048), device="cuda")

with torch.autograd.profiler.profile(use_cuda=True) as prof:
    y1 = torch.square(x)
    y2 = x * x
    y3 = torch.pow(x, 2)

```



```
print(prof.key_averages().table(
    sort_by="cuda_time_total",
    row_limit=20
))
```

This code helps discover whether the operations lower to the same or different kernels.
The main engineering lesson is:

Do not assume that mathematical equivalence implies implementation equivalence. Profile the actual program.

7.3 PyTorch Profiler with Chrome Trace

```
import torch
from torch.profiler import profile, ProfilerActivity

x_cpu = torch.randn((1024, 1024))

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True
) as prof:
    x = x_cpu.to("cuda")
    y = torch.square(x)

prof.export_chrome_trace("square_trace.json")
```

This trace can show:

- the CPU-side call to move data,
- the CUDA memory copy,
- the call from PyTorch into ATen,
- the CUDA launch,
- the GPU-side elementwise kernel.

For performance diagnosis, this helps separate three categories:

$$T_{\text{total}} = T_{\text{copy}} + T_{\text{dispatch}} + T_{\text{kernel}}.$$

7.4 Using `load_inline` for a C++ Function

The lecture introduces `load_inline` as an easy path for loading C++ code into Python.

```
from torch.utils.cpp_extension import load_inline

cpp_source = """
#include <string>

std::string hello_world() {
    return "hello world";
}
```

```

}
"""

module = load_inline(
    name="hello_world_extension",
    cpp_sources=cpp_source,
    functions=["hello_world"],
    verbose=True
)

print(module.hello_world())

```

This hides much of the complexity of:

- creating Python bindings,
- invoking a C++ compiler,
- generating build files,
- setting include paths,
- linking against PyTorch,
- importing the compiled extension.

The lecture notes that `load_inline` generates files such as a C++ source file and a build script. These generated files are educational because they show how Python bindings and build systems work without requiring the learner to write all boilerplate manually.

7.5 Custom CUDA Kernel Integrated with PyTorch

A representative CUDA source for elementwise square is:

```

#include <torch/extension.h>

__global__ void square_kernel(
    const float* input,
    float* output,
    int n
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        float x = input[idx];
        output[idx] = x * x;
    }
}

torch::Tensor square_cuda(torch::Tensor input) {
    auto output = torch::empty_like(input);

    int n = input.numel();
    int threads = 256;
    int blocks = (n + threads - 1) / threads;

```

```

square_kernel<<<blocks, threads>>>(
    input.data_ptr<float>(),
    output.data_ptr<float>(),
    n
);

return output;
}

```

The C++ binding source can expose this function to Python:

```

#include <torch/extension.h>

torch::Tensor square_cuda(torch::Tensor input);

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("square_cuda", &square_cuda, "Square tensor using CUDA");
}

```

The Python loader:

```

from torch.utils.cpp_extension import load_inline

cpp_source = """
#include <torch/extension.h>

torch::Tensor square_cuda(torch::Tensor input);

PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("square_cuda", &square_cuda, "Square tensor using CUDA");
}
"""

cuda_source = """
#include <torch/extension.h>

__global__ void square_kernel(
    const float* input,
    float* output,
    int n
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n) {
        float x = input[idx];
        output[idx] = x * x;
    }
}

torch::Tensor square_cuda(torch::Tensor input) {
    auto output = torch::empty_like(input);

    int n = input.numel();
    int threads = 256;

```

```

    int blocks = (n + threads - 1) / threads;

    square_kernel<<<blocks, threads>>>(
        input.data_ptr<float>(),
        output.data_ptr<float>(),
        n
    );

    return output;
}
"""

module = load_inline(
    name="square_cuda_extension",
    cpp_sources=cpp_source,
    cuda_sources=cuda_source,
    functions=["square_cuda"],
    extra_cuda_cflags=["-O2"],
    verbose=True
)

```

Key hardware implications:

- Each CUDA thread processes one element.
- The memory loads and stores are contiguous if the input tensor is contiguous.
- The kernel uses global memory but no shared memory.
- The operation has very low arithmetic intensity.
- Performance depends heavily on memory bandwidth, launch configuration, and tensor size.

7.6 Important Assumptions in the Custom CUDA Kernel

The simple kernel assumes:

1. The tensor is on the CUDA device.
2. The tensor dtype is FP32.
3. The tensor is contiguous or at least can be safely interpreted through linear indexing.
4. The output shape matches the input shape.
5. One thread per element is sufficient.

A production kernel would need stronger checks:

```

TORCH_CHECK(input.is_cuda(), "input must be a CUDA tensor");
TORCH_CHECK(input.dtype() == torch::kFloat32, "input must be FP32");
TORCH_CHECK(input.is_contiguous(), "input must be contiguous");

```

Without these checks, passing a CPU tensor, non-FP32 tensor, or non-contiguous tensor may produce incorrect behavior or crashes.

7.7 Triton Version of Elementwise Square

The lecture also introduces Triton as a Python DSL for writing GPU kernels.

A representative Triton square kernel is:

```
import torch
import triton
import triton.language as tl

@triton.jit
def square_kernel(x_ptr, y_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)

    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask)
    y = x * x

    tl.store(y_ptr + offsets, y, mask=mask)

def square_triton(x):
    y = torch.empty_like(x)
    n_elements = x.numel()

    grid = lambda meta: (
        triton.cdiv(n_elements, meta["BLOCK_SIZE"]),
    )

    square_kernel[grid](
        x,
        y,
        n_elements,
        BLOCK_SIZE=1024
    )

    return y
```

Explanation:

- `tl.program_id(axis=0)` gives the Triton program instance id.
- Each program processes a block of elements.
- `tl.arange(0, BLOCK_SIZE)` creates vector offsets within the block.
- `mask` prevents out-of-bounds loads and stores.
- `tl.load` reads a block of values from global memory.
- `x * x` performs vectorized elementwise multiplication.
- `tl.store` writes the result back to global memory.

The Triton programming model is block-oriented rather than explicitly thread-oriented. The programmer describes operations over blocks of elements, and Triton lowers the code toward GPU execution.

7.8 Triton Debugging with Interpret Mode

The lecture highlights Triton's interpret mode as a major usability improvement.

Representative usage:

```
@triton.jit(interpret=True)
def debug_kernel(x_ptr, y_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)

    breakpoint()

    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = x * x
    tl.store(y_ptr + offsets, y, mask=mask)
```

The key idea is that Triton kernels are otherwise black boxes during execution. In ordinary CUDA, printing internal kernel variables is inconvenient. Often one must write intermediate values to global memory and inspect them later. Triton interpret mode allows Python-style inspection for educational debugging.

7.9 Inspecting Triton PTX

Triton does not generate CUDA C++ source code. It lowers through compiler IRs and can generate PTX, which is NVIDIA's low-level virtual instruction set.

A simplified PTX-style multiply might look conceptually like:

```
mul.rn.f32    %f9, %f1, %f1;
st.global.f32 [%rd_out], %f9;
```

The lecture shows that inspecting PTX can reveal:

- how many registers are used,
- whether operations are global loads or stores,
- where the multiplication occurs,
- whether values are stored in registers,
- how the compiler maps high-level block operations to lower-level instructions.

The speaker's practical recommendation is not that every beginner must master PTX immediately. Instead, PTX can be used as an educational tool when trying to understand what Triton or a compiler generated.

7.10 `torch.compile` as a Kernel Generation Tool

The lecture explains that `torch.compile` can generate Triton kernels from PyTorch code.

Representative code:

```
import torch

def square_fn(a):
    return torch.square(a)

compiled_square = torch.compile(square_fn)

x = torch.randn((1024, 1024), device="cuda")
y = compiled_square(x)
```

To inspect generated code, one can use logging:

```
TORCH_LOGS=output_code python square.py
```

A useful experiment is:

```
import torch

def two_squares(a):
    a = torch.square(a)
    a = torch.square(a)
    return a

compiled_fn = torch.compile(two_squares)

x = torch.randn((1024, 1024), device="cuda")
y = compiled_fn(x)
```

If fusion occurs, the generated Triton code should contain both square operations inside a single kernel rather than launching two separate kernels.

The lecture describes compiler lowering at a high level:

PyTorch op $\rightarrow$ simpler primitive op $\rightarrow$ generated Triton code $\rightarrow$ compiled GPU kernel.

For example:

`torch.square(a)` $\rightarrow$ `torch.mul(a, a)` $\rightarrow$ Triton multiply kernel.

8 Performance Intuition

8.1 Why `a * a` Can Beat `torch.square(a)`

Although both compute a^2 , the profiler can show different dispatch paths. A direct multiplication operation may map to a highly optimized elementwise multiplication kernel, while square may lower through a more general power operation.

General power is more flexible:

$$y = x^p,$$

but square is a special case:

$$y = x^2 = x \cdot x.$$

A more general implementation may have more overhead or less specialized optimization. The lecture's lesson is practical: profile rather than assume.

8.2 Why a Custom Kernel Can Be Slower Than PyTorch

A custom CUDA or Triton kernel is not automatically faster. It can be slower if:

- launch parameters are poor,
- block size is too small or too large,
- the grid is too small to fill the GPU,
- memory accesses are not coalesced,
- register pressure reduces occupancy,
- unnecessary memory traffic is introduced,
- PyTorch already uses a highly optimized kernel,
- the operation is too simple to benefit from custom code.

The lecture includes a case where an initial Triton square kernel was slower than native PyTorch. After changing the block size to a larger value such as 1024, the performance trend improved significantly. This illustrates that kernel launch configuration is not cosmetic. It directly controls how much parallel work is exposed to the GPU.

8.3 Choosing Block Size

For one-element-per-thread CUDA kernels, common block sizes include:

128, 256, 512, 1024.

In Triton, `BLOCK_SIZE` controls how many elements each program instance processes. A poor block size can lead to:

- too many tiny programs,
- too few programs,
- poor memory transaction efficiency,
- poor occupancy,
- bad compiler choices,
- increased overhead relative to useful work.

There is no universally optimal block size. It depends on:

- tensor size,
- dtype,

- operation complexity,
- memory bandwidth,
- GPU architecture,
- compiler behavior,
- register use.

8.4 How Nsight Compute Guides Optimization

Nsight Compute can give hints such as:

- the grid is too small,
- occupancy is low,
- memory throughput is low,
- compute throughput is low,
- there are long scoreboard stalls,
- there is a tail effect,
- launch configuration wastes SM resources.

A practical interpretation is:

If `ncu` says the grid is too small, first try exposing more parallel work through padding, larger inputs, better launch configuration, or batching. Do not immediately rewrite everything in CUDA.

8.5 Long Scoreboard Stalls

A long scoreboard stall often means a warp is waiting for a long-latency memory operation to complete. This can happen when the kernel depends on data loaded from global memory.

A simplified model:

$$T_{\text{warp}} = T_{\text{issue}} + T_{\text{wait for memory}} + T_{\text{compute}}.$$

If memory latency is not hidden by other active warps, the SM may stall.

Ways to reduce or hide this include:

- increase occupancy,
- improve memory coalescing,
- use shared memory when there is reuse,
- increase arithmetic intensity,
- fuse operations to do more work per load,

- prefetch where appropriate,
- avoid dependent memory access chains.

For elementwise square, shared memory usually does not help because there is no reuse:

A_i is loaded once and used once.

Fusion is more promising because it increases the amount of computation per memory load.

8.6 When to Move from PyTorch to Triton to CUDA

The lecture suggests a practical escalation path:

1. Start with ordinary PyTorch.
2. Use PyTorch profiling to find bottlenecks.
3. Try `torch.compile` and check whether fusion helps.
4. Use Triton for custom fused kernels or custom memory patterns.
5. Use CUDA C++ when Triton does not expose enough control.
6. Use Nsight Compute to determine whether the remaining bottleneck is under your control.

The important idea is that profiling should decide the level of complexity. One should not begin with CUDA C++ unless there is evidence that the performance problem requires it.

9 Hardware-Level View of the Profiling Results

9.1 CPU Timeline Versus GPU Timeline

The PyTorch visual profiler separates CPU activity from GPU activity.

CPU-side work includes:

- Python function calls,
- PyTorch dispatcher overhead,
- ATen operator setup,
- CUDA runtime API calls,
- kernel launch calls.

GPU-side work includes:

- memory copies,
- elementwise kernels,
- generated Triton kernels,

- compiled CUDA kernels.

A kernel launch has a CPU-side component and a GPU-side component:

$$T_{\text{launch path}} = T_{\text{Python}} + T_{\text{dispatcher}} + T_{\text{CUDA runtime}} + T_{\text{GPU execution}}.$$

A visual trace helps show which component dominates.

9.2 Global Memory and Caches

For the square kernel, each thread performs:

global load $\rightarrow$ register multiply $\rightarrow$ global store.

The memory hierarchy involved is:

GPU DRAM $\leftrightarrow$ L2 cache $\leftrightarrow$ L1 cache $\leftrightarrow$ registers.

Nsight Compute can report L1 and L2 throughput. For a streaming pointwise operation, high memory throughput is usually desirable. Low throughput may indicate poor launch configuration, insufficient parallelism, or inefficient memory access.

Visual Concept

Draw a long tensor in global memory. Show contiguous chunks flowing through L2 and L1 into registers inside many SMs. Each thread multiplies one register value by itself and writes the result back to global memory.

9.3 Registers

In both CUDA and Triton-generated PTX, values loaded from memory are held in registers before arithmetic.

For square:

$$r_x \leftarrow A_i, \quad r_y \leftarrow r_x \cdot r_x, \quad B_i \leftarrow r_y.$$

Registers are the fastest storage available to a thread, but excessive register use can reduce occupancy because each SM has a finite register file.

9.4 Shared Memory

The simple square kernel does not need shared memory. Shared memory is useful when threads within a block reuse data. For example, tiled matrix multiplication benefits from shared memory because a tile of A and B is reused by many multiply-add operations.

For elementwise square, reuse is:

$$\text{reuse}(A_i) = 1.$$

Therefore, shared memory would likely add overhead without benefit.

10 Common Misconceptions

10.1 Misconception 1: Python Timing Is Enough

Incorrect. Since CUDA is asynchronous, Python timing often measures only launch overhead.

Correct view:

Use CUDA events or explicit synchronization for GPU timing.

10.2 Misconception 2: A Custom CUDA Kernel Is Automatically Faster

Incorrect. A custom kernel can easily be slower than PyTorch if PyTorch already uses optimized kernels or if the custom launch configuration is poor.

Correct view:

Custom kernels are useful only when they improve a measured bottleneck.

10.3 Misconception 3: Mathematical Equivalence Means Same Performance

Incorrect. The expressions

$$a^2 \quad \text{and} \quad a \cdot a$$

are mathematically equivalent, but `torch.pow(a, 2)`, `torch.square(a)`, and `a * a` may dispatch differently.

Correct view:

Profile the actual expression used in the program.

10.4 Misconception 4: Triton Generates CUDA C++

Incorrect. Triton is a Python DSL that lowers toward lower-level compiler IR and PTX. It does not simply emit CUDA C++ source.

Correct view:

Triton is a GPU kernel DSL, not a CUDA C++ code generator.

10.5 Misconception 5: Nsight Compute Is Only for Experts

Incorrect. Even without understanding every metric, `ncu` can provide useful hints about occupancy, grid size, memory throughput, and stalls.

Correct view:

Use profiler hints as a map for what to learn next.

10.6 Misconception 6: Shared Memory Always Makes Kernels Faster

Incorrect. Shared memory helps when there is data reuse. For pure streaming pointwise operations, it usually does not help.

Correct view:

Use shared memory when it reduces global memory traffic or improves reuse.

10.7 Misconception 7: `torch.compile` Is Magic

Incorrect. `torch.compile` performs graph capture, lowering, code generation, and fusion where possible. It can improve performance, but it can also generate kernels that are not faster for a specific case.

Correct view:

`torch.compile` is a compiler tool whose output should be inspected and profiled.

11 Practical Takeaways

11.1 A Practical Profiling Ladder

The lecture’s practical profiling ladder is:

1. Use CUDA events for basic timing.
2. Use the PyTorch autograd profiler to inspect operator-level behavior.
3. Use the PyTorch profiler and Chrome traces to see CPU and GPU timelines.
4. Use `torch.compile` logging to inspect generated Triton code.
5. Use Triton interpret mode to debug Triton kernels.
6. Use Triton cache artifacts and PTX to inspect low-level generated code.
7. Use Nsight Compute for occupancy, memory, launch, and stall analysis.

11.2 The Engineering Decision Rule

Before writing a custom kernel, ask:

1. Is the operation actually a bottleneck?
2. Is time spent in launch overhead, memory transfer, or GPU execution?
3. Can `torch.compile` fuse it?
4. Can a Triton kernel express the desired optimization?
5. Is CUDA C++ necessary for finer control?
6. Does Nsight Compute show a bottleneck that custom CUDA can fix?

11.3 Use Generated Code as a Teacher

A strong practical idea from the lecture is to use generated code as an educational resource.

Examples:

- Use `load_inline` and inspect generated C++ binding and build files.
- Use `TORCH_LOGS=output_code` to inspect generated Triton code.
- Use the Triton cache to inspect PTX.
- Use PyTorch’s source code to inspect real elementwise CUDA kernels.

This avoids learning only from toy textbook kernels and helps connect low-level GPU programming to actual machine learning code.

12 Project or Experiment Ideas

12.1 Experiment 1: Compare Square Implementations

Implement and benchmark:

1. `torch.square(x)`
2. `x * x`
3. `torch.pow(x, 2)`
4. custom CUDA square kernel
5. Triton square kernel
6. `torch.compile` version

Vary:

$$N \in \{2^{10}, 2^{14}, 2^{18}, 2^{22}, 2^{26}\}.$$

Measure:

- latency,
- effective bandwidth,
- number of kernels launched,
- generated code,
- Nsight Compute occupancy.

Effective bandwidth can be estimated as:

$$B_{\text{effective}} = \frac{8N}{T_{\text{seconds}}}.$$

12.2 Experiment 2: Fusion Benchmark

Compare eager versus compiled execution for:

```
def f(x):  
    x = x * x  
    x = x + 1.0  
    x = torch.relu(x)  
    x = x * 0.5  
    return x
```

Expected eager memory traffic is much higher than fused memory traffic. Use the profiler to verify whether `torch.compile` produces one fused Triton kernel.

12.3 Experiment 3: Triton Block Size Sweep

Benchmark a Triton square kernel with:

$$\text{BLOCK_SIZE} \in \{128, 256, 512, 1024, 2048\}.$$

For each block size, measure:

- latency,
- effective bandwidth,
- occupancy,
- register usage,
- Nsight Compute warnings.

The goal is to see how launch configuration affects performance.

12.4 Experiment 4: Host-to-Device Copy Cost

Compare:

```
x = torch.randn((4096, 4096))  
y = torch.square(x.to("cuda"))
```

against:

```
x = torch.randn((4096, 4096), device="cuda")  
y = torch.square(x)
```

Use Chrome traces to separate:

$$T_{\text{copy}} \quad \text{from} \quad T_{\text{kernel}}.$$

12.5 Experiment 5: Inspect `torch.compile` Output

Create small functions:

```
def f1(x):  
    return x * x  
  
def f2(x):  
    return torch.sin(x) + torch.cos(x)  
  
def f3(x):  
    return torch.relu(x * x + 1.0)
```

Run each through `torch.compile` and inspect generated code using:

```
TORCH_LOGS=output_code python experiment.py
```

Study:

- whether operations fuse,
- how variable names appear,
- whether generated Triton code is readable,
- how masks and block sizes are chosen.

12.6 Experiment 6: Nsight Compute Report Study

Run:

```
ncu --set full -o square_report python square_bench.py
```

Study:

- achieved occupancy,
- memory throughput,
- L1 and L2 throughput,
- grid size,
- block size,
- stall reasons,
- source-level correlation.

Write a short report explaining whether the kernel is launch-bound, memory-bound, occupancy-limited, or affected by tail effects.

13 Final Mental Model

The lecture's central mental model is:

A PyTorch operation is not just a Python line. It is a chain of dispatch, memory movement, kernel launches, generated code, and GPU execution. To optimize it, first make the chain visible. Use PyTorch profilers to see what runs, CUDA events to time it correctly, `torch.compile` and Triton to generate or write fused kernels, and Nsight Compute to understand whether the GPU is actually being used well.

For the simple square operation, the full stack is:

Python expression $\rightarrow$ ATen operator $\rightarrow$ CUDA launch $\rightarrow$ GPU kernel $\rightarrow$ global load $\rightarrow$ register multiply $\rightarrow$ global

The engineering lesson is:

Optimization should be profiler-driven, not mythology-driven.

Start with evidence. If PyTorch is fast enough, keep PyTorch. If fusion is needed, try `torch.compile`. If custom memory behavior is needed, try Triton. If full hardware control is needed, use CUDA C++. At every stage, profile again.